

# fn then\_saturating\_cast

Michael Shoemate (@Shoeboxam)

This proof resides in “**contrib**” because it has not completed the vetting process.

Proves soundness of the implementation of `then_saturating_cast` in `mod.rs` at commit `f5bb719` (outdated<sup>1</sup>).

## 1 Hoare Triple

### Precondition

#### Compiler-verified

- Generic T0 implements trait `SaturatingCast<IBig>`

#### Caller-verified

None

### Pseudocode

```
1 def then_saturating_cast() -> Function[Vec[IBig], Vec[T0]]:  
2   return Function.new(lambda x: [T0.saturating_cast(x_i) for x_i in x])
```

### Postcondition

**Theorem 1.1.** For every setting of the input parameters (T0) to `then_saturating_cast` such that the given preconditions hold, `then_saturating_cast` raises an error (at compile time or run time) or returns a valid postprocessor. A valid postprocessor has the following property:

1. (Data-independent errors). For every pair of members  $x$  and  $x'$  in `input_domain`, `function(x)`, `function(x')` either both raise the same error, or neither raise an error.

*Proof.* Since `T0.saturating_cast` is infallible, the function is infallible, meaning that the function cannot raise data-dependent errors. Therefore the function is a valid postprocessor.  $\square$

---

<sup>1</sup>See new changes with `git diff f5bb719..3bd9f45 rust/src/measurements/noise/nature/integer/mod.rs`